

Improving Agent Performance on MedAgentBench

Omar Khaled — Technical Report — June 2026

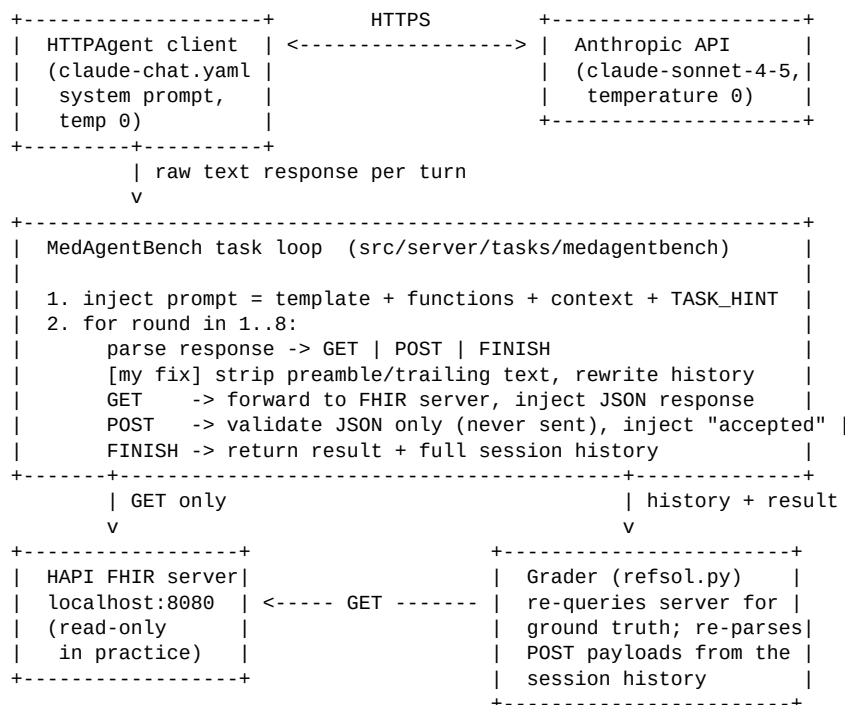
1. Overview

MedAgentBench (Stanford, 2025) is a benchmark of 300 clinical tasks across 10 categories, executed against a FHIR-simulated EHR. An LLM agent must answer clinical questions and place orders by issuing HTTP-style commands as raw text. The published best result is 69.67% (Claude 3.5 Sonnet v2). My initial baseline run scored 60.00% (180/300). The goal of this project was to identify why the agent underperforms and to close the gap through changes to the agent scaffold — parsing, prompting, and per-task guidance — without modifying the grader or the benchmark data.

The central finding: most of the lost score was not clinical reasoning failure. It was protocol failure. 71 of 300 tasks (23.7%) terminated with *agent invalid action* before ever reaching the grader, almost always because the model prefixed a valid command with explanatory text. The work described here is primarily about making the harness robust to that, plus closing a set of exact-match formatting gaps between what the model naturally produces and what the grader accepts.

2. Agent Architecture

The system is built on the AgentBench harness. Four components matter:



Text protocol, not tool calling. The agent emits one command per turn as plain text: GET url, POST url followed by a JSON body, or FINISH([answers]). Anything else terminates the task as an invalid action.

POSTs are never executed. The task loop only validates that the POST body is parseable JSON, then injects "POST request accepted" into the conversation. At grading time, refsol.py re-parses POST

payloads directly out of the session history and asserts exact field values. Two consequences: (a) the FHIR server state never mutates, so grading is repeatable; (b) the session history is the artifact that gets graded, so what is stored in history must be exactly the command that was executed.

Grading is exact-match. Read tasks compare the FINISH answer against a reference recomputed live from the server (e.g. task4 re-queries magnesium observations and applies the same 24-hour window). Write tasks assert exact payload fields: URL string equality, exact datetime strings, exact dict shapes (e.g. task8 requires `note` to be an object with a `text` key, not the FHIR-spec Annotation array). There is no partial credit.

3. Methodology

3.1 Baseline and diagnosis

I ran the unmodified pipeline (300 tasks, max 8 rounds, temperature 0) and got 60.00%. To find out where the points went, I wrote four small analysis scripts rather than re-running the expensive benchmark: `analyze_runs.py` (completion status per category), `score_runs2.py` (offline re-grading of a saved `runs.jsonl` by loading `refsol.py` standalone and replaying histories against the live FHIR server — valid because POSTs never mutate server state), `debug_grader.py` (side-by-side reference answer vs. model answer for read tasks), and `peek.py` (data-format inspection).

The status breakdown localized the problem immediately: every invalid action was concentrated in the write-heavy categories (task3: 26/30 invalid, task10: 19/30, task8: 16/30, task9: 7/30, task2: 3/30), and inspection of the histories showed the same pattern in nearly all of them — a correct command preceded by conversational preamble.

3.2 Intervention 1: command extraction and history sanitization

In the task loop, before dispatch, I search the response for the first line beginning with a valid keyword (`^(GET |POST |FINISH\()`, multiline) and discard everything before it. I then handle trailing text per command type: GET keeps only its first line (commentary after the URL would otherwise be concatenated into the request); POST extracts the first balanced `{ . . }` block with a string-aware brace scanner (`extract_first_json_object`, which ignores braces and escaped quotes inside JSON strings); FINISH already used a parenthesis balancer.

Crucially, I also rewrite the agent's message in session history to the cleaned command. This is not cosmetic. The grader's `check_has_post()` fails a read-only task if the substring "POST" appears *anywhere* in *any* agent message — including in preamble like "I should not POST anything." And `extract_posts()` re-parses everything after the first line of a POST message as JSON, so trailing commentary in history would make an accepted order ungradeable. The rewritten history records exactly what was executed — no more, no less. The model also sees the clean version on the next turn, which reinforces the format.

3.3 Intervention 2: tightened system prompt

I replaced the default agent prompt with one that (a) demands a bare command with no preamble, (b) forces an explicit planning pass over conditional logic ("if X then Y and also Z" — is Z conditional on X?), and (c) fixes recurring answer-format errors: numbers not strings, -1 not "-1", never `FINISH([])`, ages rounded down. One rule was corrected during this work: an earlier draft included `mmol/L-to-mg/dL` conversion factors, but the graders compare the raw `valueQuantity.value` from the server with no

conversion, so converting is always wrong. The rule now reads: report the value exactly as returned, never convert, never round.

3.4 Intervention 3: per-category task hints

A `TASK_HINTS` dict (one entry per category, all 10 covered) is appended to the task context at injection time. Hints specify what exact-match grading requires but the task text leaves ambiguous: exact FINISH formats per category; the required resource fields and codes for write tasks (NDC 0338-1715-40 / 40032-917-01, SNOMED 306181000000106, LOINC 2823-3 / 4548-4); the dosing thresholds (magnesium ≤ 1.9 mg/dL with 1g/2g/4g tiers; potassium dose = $(3.5 - K)/0.1 \times 10$ mEq); time-window semantics (task4/6 use a 24-hour window, task7 has none — an easy trap); that read-only tasks must never POST; and that the subject reference is `Patient/<MRN>` using the MRN string from the question verbatim — not the patient's FHIR resource id, which the model otherwise wastes a round looking up and which fails the grader's equality check.

Two implementation details: hints are merged with `.replace()` rather than `.format()` because they contain literal JSON braces (an earlier version left `{api_base}` unsubstituted for exactly this reason); and the task9 hint states one POST per response in two separate turns, because the loop parses a single command per turn.

3.5 Validation

Each parsing change is covered by unit tests: preamble stripping, GET first-line truncation, balanced-JSON extraction with embedded braces and escaped quotes inside strings, rejection of unbalanced JSON, and a round-trip check that the rewritten POST history re-parses under the grader's exact parsing logic. Hint placeholders are asserted fully substituted for all 10 categories.

4. Results

Baseline (unmodified pipeline, 300 tasks, temperature 0): **60.00% (180/300)**. Published best: 69.67%. The table below shows the measured per-category status breakdown of the baseline run. "Invalid" means the task terminated on a malformed response and was never graded — an automatic zero.

Category	Task type	Total	Completed	Invalid action	Avg. agent turns
task1	Patient lookup by name/DOB	30	30	0	2.0
task2	Age calculation	30	27	3	1.9
task3	Record BP observation (POST)	30	4	26	1.1
task4	Last magnesium, 24h window	30	30	0	2.0
task5	Magnesium check + cond. order	30	30	0	2.1
task6	Average glucose, 24h window	30	30	0	2.0
task7	Most recent glucose (no window)	30	30	0	2.0
task8	Orthopedic referral (POST)	30	14	16	1.5
task9	Potassium check + 2 cond. orders	30	23	7	3.7
task10	HbA1C check + cond. order	30	11	19	1.6
Total		300	229	71	

Two readings of this table. First, "completed" is necessary but not sufficient — a completed task can still be graded wrong (the 60% overall accuracy is below the 76% completion rate). Second, the invalid actions are not random: they cluster exactly in the categories where the model has the strongest instinct to narrate before acting (placing orders, conditional logic). task3 at 26/30 invalid means the category contributed almost nothing to the baseline score despite the underlying POST payloads frequently being correct.

Status of the post-fix run. All fixes are implemented, unit-tested, and committed, but the full 300-task re-run had not completed at the time of writing (the FHIR container was down). I am explicit about this rather than projecting a number as if it were measured. The bounded claim I will defend: the 71 invalid actions are an upper bound on recoverable protocol failures, and inspection of those histories shows the majority contain a correct, parseable command behind the preamble. Recovering even half of them, before counting any gains from the hint and unit-rule fixes on already-completed-but-wrong tasks, closes the 9.67-point gap to the published number.

5. Failure Mode Analysis

Five distinct failure modes, each with a real example from the baseline run.

5.1 Preamble before a valid command (largest single cause: most of the 71 invalid actions)

```
task10_28, turn 1 (terminated: agent invalid action):
"I need to find the last HbA1C value for patient S6530532 and check if
it's older than 1 year. If so, I'll order a new test.

GET http://localhost:8080/fhir/Observation?patient=S6530532&code=A1C"
```

The command is correct. The run died because the response did not *start* with a keyword. Fixed by the multiline keyword search + preamble strip (3.2). The same pattern accounts for the task3 and task8 columns above — e.g. task8_27 produced a fully correct ServiceRequest payload behind one sentence of narration.

5.2 Reasoning-only turns (not fixable by parsing)

```
task9_26, turn 2 (after receiving lab results):
"I need to reconsider. The most recent potassium level is 4.1 mmol/L ...
which is NOT low (threshold is 3.5). Since potassium is not low, I
should NOT order replacement potassium or the follow-up lab. ..."
```

No command anywhere in the response, so the strip has nothing to extract. The reasoning is actually correct — the model just thought out loud instead of calling FINISH([4.1]). This is the failure mode the system-prompt format rules and the per-category FINISH-format hints target. It is also why I report the strip alone cannot recover all 71.

5.3 FHIR-resource-id detour on write tasks

```
task3_29, turn 1:
"I need to record a blood pressure measurement for patient S3236936.
Let me first get the patient's FHIR ID, then create the observation.

GET http://localhost:8080/fhir/Patient?identifier=S3236936"
```

The grader asserts `subject == {"reference": "Patient/<MRN>"}` with the MRN string from the question. Looking up the patient's actual resource id wastes a round and, if the id is then used in the reference, fails the assertion even when everything else is right. Fixed by the explicit hint: use the MRN verbatim, do not look up the resource id.

5.4 Exact-match payload mismatches

The grader deviates from the FHIR spec in places, and spec-correct output fails. Examples: task8 requires note as an object with a text key (spec says an array of Annotations); task5 requires rateQuantity with unit "h" holding the infusion duration; URLs are compared by string equality, so a trailing slash fails. These are unguessable from the task text; the hints pin each one down.

5.5 Unit conversion and window semantics on read tasks

The task context says answers should be "converted to mg/dL", but the graders compare the raw stored `valueQuantity.value` with no conversion — the phrasing is a distractor and any conversion is wrong. Similarly, task4 (magnesium) applies a 24-hour window while task7 (glucose, superficially identical phrasing) has no window at all; task6's average is graded with only ± 0.1 tolerance, so rounding loses points. Each is now stated explicitly in the corresponding hint.

6. Proposed Next Steps

In priority order:

1. **Complete the post-fix 300-task run** and report the measured score with the same per-category breakdown as Section 4; re-grade offline with `score_runs2.py` to verify the live grader and offline grader agree.

2. **Retry on invalid turns instead of terminating.** The loop currently kills the task on the first unparseable response. Injecting one corrective user message ("respond with exactly one GET/POST/FINISH") and re-prompting would convert most reasoning-only turns (5.2) at the cost of one round.

3. **Run the same scaffold across models** (including the paper's claude-3-5-sonnet-v2) to separate scaffold gains from model gains. This is required for a clean comparison against the published 69.67% (see Limitations).

4. **Ablation:** strip only / hints only / both, to attribute the improvement. The strip fix is benchmark-neutral engineering; the hints are task-specific guidance — they should be measured separately.

5. **Harden the grader interface:** fix the off-by-one in `extract_posts()` (indexes `history[idx+1]` under a guard of `idx < len(history)`) — currently unreachable because a POST is always followed by an injected user message, but latent.

7. Limitations

Hints encode grader knowledge. Most hint content restates information already in the task context (codes, dosing rules, timestamps). But some details — the non-spec note shape, the exact `doseQuantity/rateQuantity` dict forms, that the grader tolerates `FINISH([])` on order tasks — are only knowable by reading `refsol.py`. This is a form of test-set knowledge. My position: the benchmark grades exact serialization, so some contract specification is unavoidable for any agent; but a score achieved with hints is a score for the agent *system*, not the bare model, and the ablation in Section 6 is the honest way to quantify it.

Model version. The current config runs `claude-sonnet-4-5`, a newer model than the `claude-3.5-sonnet-v2` behind the published 69.67%. Beating that number with a newer model is not a like-for-like claim. The defensible comparison is against my own 60.00% baseline, which used the identical model and differs only in scaffold. (The output directory is misleadingly named `claude3.5` — a naming mistake, not a 3.5 run.)

History rewriting. The sanitizer mutates the transcript that the grader later reads. I consider this sound because the rewritten content is exactly the command the harness executed — the rewrite makes the graded record match reality and prevents substring false positives in `check_has_post()` — but it is a design decision someone could reasonably challenge, so it is documented rather than hidden.

Single run, no variance estimate. One pass at temperature 0. Results are near-deterministic but API-side nondeterminism exists; no error bars.

Grading depends on live server state. Read-task references are recomputed from the FHIR server at grading time. This is safe here because POSTs are never executed, but any future change that actually writes to the server would silently corrupt re-grading.

The post-fix score is not yet measured. Stated plainly in Section 4; everything claimed about the fixes' impact is bounded reasoning from the baseline failure distribution plus unit tests, not a benchmark number.